**Introduction to Electronic Design Automation**

# Homework 1

**Student:** 吳亞倫
**ID:** B13901011
**Department:** 電機工程學系

## 1 Design Methodology and Design Flow

### 1. (a)

# 2 Algorithm and Complexity

## 2. (a)

Let $g_n(x) = x^n$. Then $g_n(x)$ satisfies the recurrence relation

$$g_n(x) = x \cdot g_{n-1}(x),$$

with base case

$$g_0(x) = 1.$$

By definition,

$$P_{n(x)} = P_{n-1}(x) + a_0 g_n(x).$$

For each recursive step, one multiplication is required to compute $g_n(x)$, and one multiplication together with one addition is required to compute $P_{n(x)}$. Hence, computing $P_{n(x)}$ takes $2n$ multiplications and $n$ additions overall, so the total time complexity is $O(n)$.

## 2. (b)

First, for each recursive step of calculating $P_i'(x) = xP_{i-1}'(x) + a_i$, one multiplication and one addition are required. Hence, calculating $P_{n-1}'(x)$ takes $n-1$ multiplications and $n-1$ additions overall, so the time complexity is $O(n-1)$.

Next, to calculate $P_n(x) = xP_n'(x) + a_0$, we need one multiplication and one addition. Hence, calculating $P_n(x)$ takes $n$ multiplications and $n$ additions overall, so the time complexity is $O(n)$.

# 3 NP-Completeness and Polynomial-Time Reduction

## 3. (a)

To prove that the feasibility problem of 0,1-ILP is NP-complete, it suffices to show that it is in NP and that it is NP-hard.

First, the problem is in NP. Given an assignment to the binary variables, we can verify in polynomial time whether all linear constraints are satisfied by simply substituting the assigned values into each constraint and checking its validity. Since the number of constraints is polynomial in the input size, the verification procedure runs in polynomial time.

Next, we show that the problem is NP-hard by giving a polynomial-time reduction from 3-SAT. Consider an arbitrary 3-SAT instance with variables $x_1, x_2, ..., x_n$ and clauses $C_1, C_2, ..., C_m$. We construct a corresponding 0,1-ILP instance as follows.

- For each Boolean variable $x_i$, introduce a binary variable $y_i \in \{0, 1\}$.
- For each clause $C_j = (a \vee b \vee c)$, where $a$, $b$, and $c$ are literals (can be $x_i$ or $\neg x_i$), introduce the linear constraint

  $$y_a + y_b + y_c \geq 1,$$

  where $y_a = y_i$ if $a = x_i$, and $y_a = 1 - y_i$ if $a = \neg x_i$. The definitions of $y_b$ and $y_c$ are analogous.

This construction ensures that each clause is satisfied if and only if the corresponding linear constraint is satisfied. Therefore, the original 3-SAT formula is satisfiable if and only if the

constructed 0,1-ILP instance is feasible. Moreover, the transformation clearly requires only polynomial time.

Hence, 0,1-ILP feasibility is NP-hard. Since it is both in NP and NP-hard, it follows that the feasibility problem of 0,1-ILP is NP-complete.

## 3. (b)

To prove that Circuit-SAT is NP-complete, we show that it is in NP and NP-hard.

First, Circuit-SAT is in NP. Given an assignment to the input variables, we can evaluate the circuit gate by gate in polynomial time and check whether the output is 1.

Next, we prove NP-hardness by reducing 0,1-ILP to Circuit-SAT. Consider an instance of 0,1-ILP: $A\mathbf{x} \leq \mathbf{b}$, where each $x_i \in \{0, 1\}$. That is, the instance consists of inequalities

$$a_{j1}x_1 + a_{j2}x_2 + ... + a_{jn}x_n \leq b_j,$$

for $j = 1, 2, ..., m$.

For each inequality, we build a subcircuit. Since each $x_i$ is binary, each term $a_{ji}x_i$ is either 0 or $a_{ji}$, so it can be produced by a small selector subcircuit. We then use binary adder circuits to sum all these terms and obtain the left-hand side of the inequality. After that, we use a comparator circuit to check whether this sum is at most $b_j$. Thus, the subcircuit outputs 1 if and only if the inequality is satisfied.

We repeat this construction for every inequality and connect all resulting outputs to a final AND gate. Therefore, the whole circuit outputs 1 if and only if all inequalities are satisfied, that is, if and only if the original 0,1-ILP instance is feasible.

This transformation can be carried out in polynomial time. Indeed, each selector, adder, and comparator circuit has size polynomial in the bit-length of the coefficients and constants, and the number of inequalities and variables is polynomial in the input size. Therefore, the entire circuit has polynomial size and can be constructed in polynomial time.

Hence, we have a polynomial-time reduction from 0,1-ILP to Circuit-SAT. Therefore, Circuit-SAT is NP-hard. Since it is also in NP, it follows that Circuit-SAT is NP-complete.

# 4 SAT and Integer Linear Programming

In this $k$-coloring problem, the graph $G(V, E)$ is shown as Figure 1 below, where $V = \{1, 2, 3, 4, 5, 6, 7\}$ and $E = \{(1, 2), (2, 3), (3, 4), (4, 5), (5, 1), (1, 6), (3, 6), (2, 7), (4, 7)\}$.

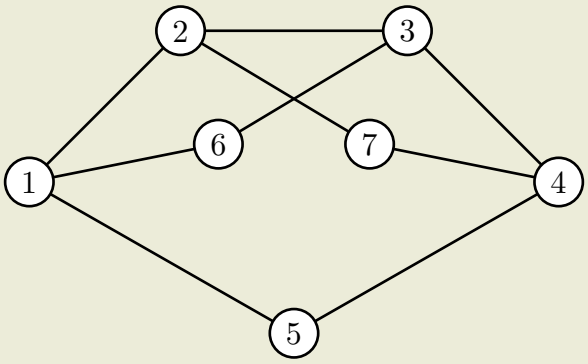


Figure 1: Graph for the $k$-coloring problem

## 4. (a)

To reduce the $k$-coloring problem to SAT, we introduce Boolean variables and construct a Boolean formula that is satisfiable if and only if the graph can be colored with k colors.

**Case** $k = 2$

First, considering $k = 2$ case. Let $c_i$ be a Boolean variable indicating the color assigned to vertex $i$, where $c_i \in \{0, 1\}$.

For every edge $(i, j) \in E$, the two endpoints must have different colors, so we require $c_i \neq c_j$. This is equivalent to the CNF formula

$$(c_i \vee c_j) \wedge (\neg c_i \vee \neg c_j).$$

Therefore, the Boolean formula for the 2-coloring problem is:

$$f_2 = \bigwedge_{(i,j)\in E} \big((c_i \vee c_j) \wedge (\neg c_i \vee \neg c_j)\big).$$

Refering to one reference article about `MiniSat`, we need to express the formula in a specific format file, so called `DIMACS CNF`. The first is the header line, which specifies the number of variables and clauses in the formula. From the 2nd line onwards, we list all the clauses in the formula. The following is my input file for the 2-coloring problem and the result is show in Figure 2 below.

**2-coloring problem MiniSat input file**

```
p cnf 7 18
1 2 0
-1 -2 0
2 3 0
-2 -3 0
3 4 0
-3 -4 0
4 5 0
-4 -5 0
5 1 0
-5 -1 0
```

```
1 6 0
-1 -6 0
3 6 0
-3 -6 0
2 7 0
-2 -7 0
4 7 0
-4 -7 0
```

```
❯ minisat 2colorsat.txt
============================[ Problem Statistics ]=============================
|                                                                             |
|  Number of variables:             7                                         |
|  Number of clauses:              18                                         |
|  Parse time:                   0.00 s                                       |
|  Eliminated clauses:           0.00 Mb                                      |
|  Simplification time:          0.00 s                                       |
|                                                                             |
===============================================================================
Solved by simplification
restarts              : 0
conflicts             : 0              (0 /sec)
decisions             : 0              ( nan % random) (0 /sec)
propagations          : 1              (434 /sec)
conflict literals     : 0              ( nan % deleted)
Memory used           : 4.52 MB
CPU time              : 0.002305 s

UNSATISFIABLE
```

Figure 2: SAT solver result for the 2-coloring problem

**Case $k = 3$**

For the $k = 3$ case, we use a different encoding. Let $c_{i,c}$ be a Boolean variable indicating whether vertex $i$ is colored with color $c$, where $c \in \{0, 1, 2\}$.

We need to ensure that each vertex is assigned at least one color, that is, for each vertex $i$, we require

$$\big(c_{i,0} \vee c_{i,1} \vee c_{i,2}\big).$$

We also need to ensure that each vertex is assigned at most one color, that is, for each vertex $i$ and for each pair of distinct colors $c$ and $c'$, we require

$$\big(\neg c_{i,c} \vee \neg c_{i,c'}\big).$$

Finally, we need to ensure that adjacent vertices have different colors. For each edge $(i, j) \in E$ and for each color $c$, we require

$$\big(\neg c_{i,c} \vee \neg c_{j,c}\big).$$

Hence, the Boolean formula for the 3-coloring problem is:

$$f_3 = \left( \bigwedge_{i \in V} (x_{i,0} \vee x_{i,1} \vee x_{i,2}) \right)$$
$$\wedge \left( \bigwedge_{i \in V} ((\neg x_{i,0} \vee \neg x_{i,1}) \wedge (\neg x_{i,0} \vee \neg x_{i,2}) \wedge (\neg x_{i,1} \vee \neg x_{i,2})) \right)$$
$$\wedge \left( \bigwedge_{(i,j) \in E} \bigwedge_{c=0}^{2} (\neg x_{i,c} \vee \neg x_{j,c}) \right)$$

I also create the input file for the 3-coloring problem ($x_{i,c}$ is assign to $3i - c$ in the DIMACS CNF format) and the result is show in Figure 3 below.

**3-coloring problem MiniSat input file**

```
p cnf 21 55
1 2 3 0
4 5 6 0
7 8 9 0
10 11 12 0
13 14 15 0
16 17 18 0
19 20 21 0
-1 -2 0
-1 -3 0
-2 -3 0
-4 -5 0
-4 -6 0
-5 -6 0
-7 -8 0
-7 -9 0
-8 -9 0
-10 -11 0
-10 -12 0
-11 -12 0
-13 -14 0
-13 -15 0
-14 -15 0
-16 -17 0
-16 -18 0
-17 -18 0
-19 -20 0
-19 -21 0
-20 -21 0
-3 -6 0
-2 -5 0
-1 -4 0
-6 -9 0
-5 -8 0
-4 -7 0
-9 -12 0
-8 -11 0
-7 -10 0
```

```
-12 -15 0
-11 -14 0
-10 -13 0
-15 -3 0
-14 -2 0
-13 -1 0
-3 -18 0
-2 -17 0
-1 -16 0
-9 -18 0
-8 -17 0
-7 -16 0
-6 -21 0
-5 -20 0
-4 -19 0
-12 -21 0
-11 -20 0
-10 -19 0
```

```
❯ minisat 3colorsat.txt
============================[ Problem Statistics ]=============================
|                                                                             |
|  Number of variables:            21                                         |
|  Number of clauses:              55                                         |
|  Parse time:                   0.00 s                                       |
|  Eliminated clauses:           0.00 Mb                                      |
|  Simplification time:          0.00 s                                       |
|                                                                             |
============================[ Search Statistics ]==============================
| Conflicts |          ORIGINAL         |          LEARNT          | Progress |
|           |    Vars  Clauses Literals |    Limit  Clauses Lit/Cl |          |
===============================================================================
===============================================================================
restarts              : 1
conflicts             : 0              (0 /sec)
decisions             : 1              (0.00 % random) (383 /sec)
propagations          : 0              (0 /sec)
conflict literals     : 0              ( nan % deleted)
Memory used           : 4.57 MB
CPU time              : 0.00261 s

SATISFIABLE
```

Figure 3: SAT solver result for the 3-coloring problem

## 4. (b)

To reduce the $k$-coloring problem to Integer Linear Programming (ILP), we introduce the following variables. For each vertex $i \in V$ and each color $c \in \{1, 2, ..., k\}$, we introduce a binary variable $x_{i,c}$, where $x_{i,c} = 1$ if vertex $i$ is colored with color $c$, and 0 otherwise.

We need to ensure that each vertex is assigned exactly one color, which can be expressed as the following constraints:

$$\sum_{c=1}^{k} x_{i,c} = 1, \forall i \in V.$$

We also need to ensure that adjacent vertices have different colors. For each edge $(i, j) \in E$ and for each color $c \in \{1, 2, ..., k\}$, we can express this constraint as:

$$x_{i,c} + x_{j,c} \leq 1.$$

The above constraints can be combined to form the ILP formulation of the $k$-coloring problem. The objective function can be arbitrary (e.g., minimize 0) since we are only interested in feasibility.

By reading the documentation of `lp_solve`, we can write the ILP formulation in its input format. The following is my input file for the 2-coloring problem and the result is show in Figure 4 below.

**2-coloring problem lp_solve input file**

```
min: 0;

x_1_1 + x_1_2 = 1;
x_2_1 + x_2_2 = 1;
x_3_1 + x_3_2 = 1;
x_4_1 + x_4_2 = 1;
x_5_1 + x_5_2 = 1;
x_6_1 + x_6_2 = 1;
x_7_1 + x_7_2 = 1;

x_1_1 + x_2_1 <= 1; x_1_2 + x_2_2 <= 1;
x_2_1 + x_3_1 <= 1; x_2_2 + x_3_2 <= 1;
x_3_1 + x_4_1 <= 1; x_3_2 + x_4_2 <= 1;
x_4_1 + x_5_1 <= 1; x_4_2 + x_5_2 <= 1;
x_5_1 + x_1_1 <= 1; x_5_2 + x_1_2 <= 1;
x_1_1 + x_6_1 <= 1; x_1_2 + x_6_2 <= 1;
x_3_1 + x_6_1 <= 1; x_3_2 + x_6_2 <= 1;
x_2_1 + x_7_1 <= 1; x_2_2 + x_7_2 <= 1;
x_4_1 + x_7_1 <= 1; x_4_2 + x_7_2 <= 1;

bin x_1_1, x_1_2, x_2_1, x_2_2, x_3_1, x_3_2, x_4_1,
    x_4_2, x_5_1, x_5_2, x_6_1, x_6_2, x_7_1, x_7_2;
```

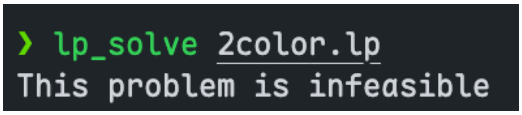


Figure 4: ILP solver result for the 2-coloring problem

The following is my input file for the 3-coloring problem and the result is show in Figure 5 below.

**3-coloring problem lp_solve input file**

```
min: 0;

x_1_1 + x_1_2 + x_1_3 = 1;
x_2_1 + x_2_2 + x_2_3 = 1;
x_3_1 + x_3_2 + x_3_3 = 1;
x_4_1 + x_4_2 + x_4_3 = 1;
x_5_1 + x_5_2 + x_5_3 = 1;
```

```
x_6_1 + x_6_2 + x_6_3 = 1;
x_7_1 + x_7_2 + x_7_3 = 1;

x_1_1 + x_2_1 <= 1; x_1_2 + x_2_2 <= 1; x_1_3 + x_2_3 <= 1;
x_2_1 + x_3_1 <= 1; x_2_2 + x_3_2 <= 1; x_2_3 + x_3_3 <= 1;
x_3_1 + x_4_1 <= 1; x_3_2 + x_4_2 <= 1; x_3_3 + x_4_3 <= 1;
x_4_1 + x_5_1 <= 1; x_4_2 + x_5_2 <= 1; x_4_3 + x_5_3 <= 1;
x_5_1 + x_1_1 <= 1; x_5_2 + x_1_2 <= 1; x_5_3 + x_1_3 <= 1;
x_1_1 + x_6_1 <= 1; x_1_2 + x_6_2 <= 1; x_1_3 + x_6_3 <= 1;
x_3_1 + x_6_1 <= 1; x_3_2 + x_6_2 <= 1; x_3_3 + x_6_3 <= 1;
x_2_1 + x_7_1 <= 1; x_2_2 + x_7_2 <= 1; x_2_3 + x_7_3 <= 1;
x_4_1 + x_7_1 <= 1; x_4_2 + x_7_2 <= 1; x_4_3 + x_7_3 <= 1;

bin x_1_1, x_1_2, x_1_3, x_2_1, x_2_2, x_2_3, x_3_1, x_3_2, x_3_3,
    x_4_1, x_4_2, x_4_3, x_5_1, x_5_2, x_5_3, x_6_1, x_6_2, x_6_3,
    x_7_1, x_7_2, x_7_3;
```

```
❯ lp_solve 3color.lp

Value of objective function: 0

Actual values of the variables:
x_1_1                         0
x_1_2                         0
x_1_3                         1
x_2_1                         0
x_2_2                         1
x_2_3                         0
x_3_1                         0
x_3_2                         0
x_3_3                         1
x_4_1                         0
x_4_2                         1
x_4_3                         0
x_5_1                         1
x_5_2                         0
x_5_3                         0
x_6_1                         1
x_6_2                         0
x_6_3                         0
x_7_1                         1
x_7_2                         0
x_7_3                         0
```

Figure 5: ILP solver result for the 3-coloring problem

# 5 Model of Computation

## 5. (a)

For the regular expression

$$(01^*0)^*1,$$

we construct a DFA as follows. State $q_0$ is the start state. Reading `0` starts a loop and moves the automaton to $q_1$, where it may read any number of `1`′ s. Reading `0` from $q_1$ completes the loop and returns to $q_0$. Reading `1` from $q_0$ corresponds to the final symbol of the string, so the automaton moves to the accepting state $q_2$.

Since a DFA must define a transition for every input symbol at every state, we add a trap state $q_t$. Any input read after reaching $q_2$ leads to $q_t$, and $q_t$ loops on both `0` and `1`.

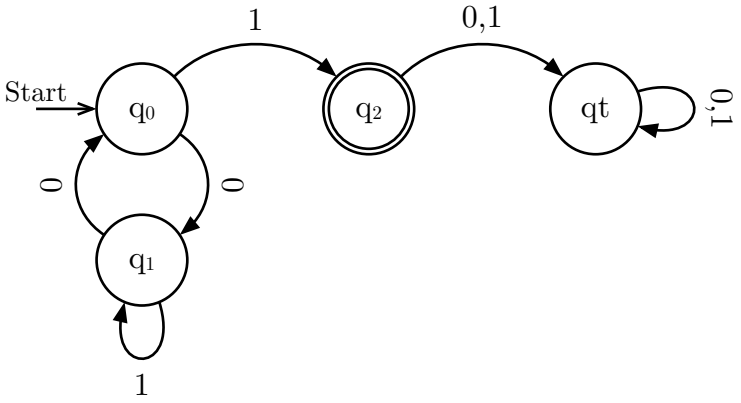


Figure 6: DFA for the regular expression $(01^*0)^*1$

## 5. (b)

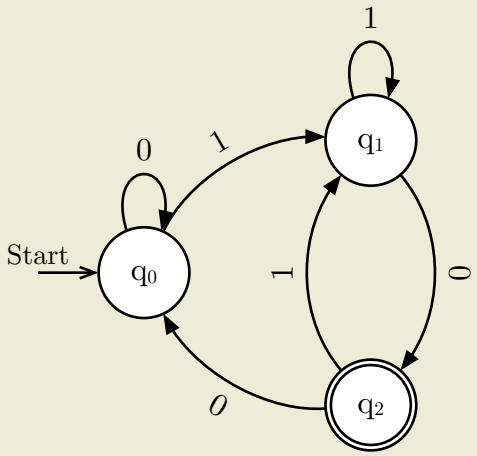


Figure 7: The finite automaton give in the problem statement

To derive the regular expression for the language accepted by the given automaton, we analyze the paths:

- First time from $q_0$ to $q_2$:
  1. zero or many `0`
  2. one or many `1`
  3. one `0`

  ▸ The regex for this part is $0^*1^+0$.

- Loop back from $q_2$ to $q_0$ to $q_2$:
  1. one or many `0`
  2. one or many `1`
  3. one `0`
  - The regex for this part is $0^+1^+0$.
- Loop back from $q_2$ to $q_1$ to $q_2$:
  1. one or many `1`
  2. one `0`
  - The regex for this part is $1^+0$.

Combining the looping back parts, we have

$$(0^+1^+0) \cup (1^+0) = 0^*1^+0,$$

so the regex for the language accepted by the automaton is

$$0^*1^+0(0^*1^+0)^* = (0^*1^+0)^+$$

## 5. (c)

The required timed automaton is as follows. The label RE stands for "Requesting" and AG stands for "Access Granted".

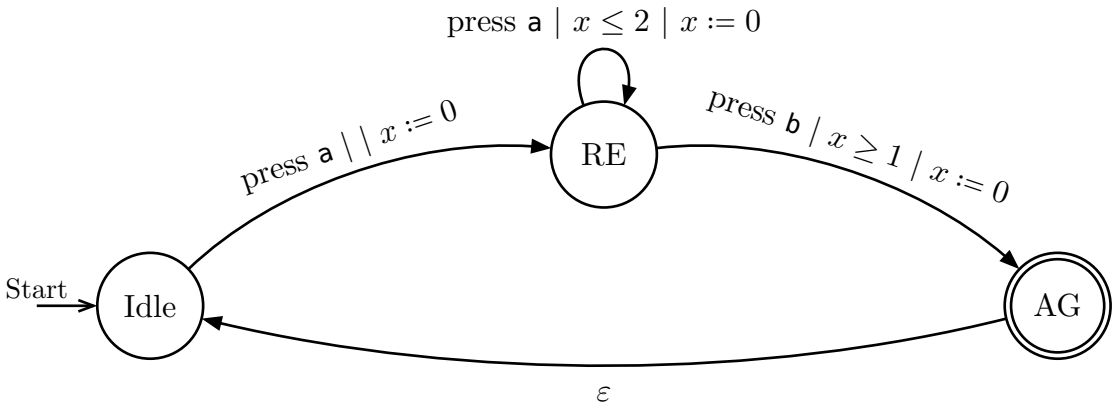


Figure 8: Timed automaton for the timed access control system

# 6 Scheduling